Lezione 1 — Automi finiti deterministici (DFA)

Elementi di Informatica Teorica · corrisponde a Sipser, Lecture 1 e §1.1 del libro

1. L'idea in breve

Un **automa finito deterministico** (DFA, *Deterministic Finite Automaton*) è il modello di calcolo più semplice che esista. Riceve in ingresso una stringa, la legge **un simbolo alla volta, da sinistra a destra**, e alla fine risponde solo "sì" (accetta) o "no" (rifiuta).

Dentro ha un numero **finito** di **stati**. In ogni momento si trova in uno solo di essi. A ogni simbolo letto passa in un altro stato (o resta nello stesso) seguendo una regola fissa. Quando la stringa è finita guarda dove si trova: se è in uno **stato di accettazione** accetta, altrimenti rifiuta.

Non può tornare indietro sulla stringa, non può scrivere, non ha una memoria extra: **tutto quello che "ricorda" è lo stato in cui si trova**. Questo è il concetto più importante della lezione, e ci tornerai per progettare qualunque automa.

> **Chiave di lettura.** Ogni stato rappresenta *un'informazione riassuntiva* sulla parte di stringa letta finora. Progettare un DFA significa chiedersi: "*che cosa mi basta ricordare, al minimo, per decidere alla fine?*"

2. Un primo esempio

Vogliamo riconoscere le stringhe sull'alfabeto {0, 1} che contengono un **numero pari di 1** (zero compreso). Cosa serve ricordare? Solo se, finora, gli 1 visti sono pari o dispari. Bastano quindi due stati.

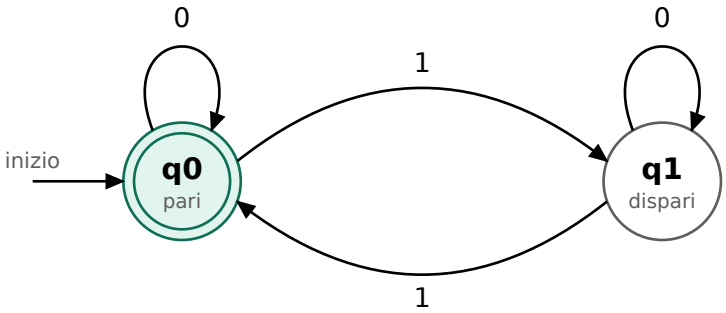


Figura 1. DFA M_1 per le stringhe con un numero pari di 1. Il doppio cerchio indica lo stato di accettazione.

Come si legge il disegno:

- ogni **cerchio** è uno stato; la freccia "inizio" indica lo **stato iniziale**;
- il **doppio cerchio** indica uno stato di **accettazione**;
- una **freccia** da p a q con etichetta a significa: "se sono in p e leggo a, vado in q";
- una freccia che torna sullo stesso stato (*cappio*) significa che con quel simbolo si resta dove si è.

Qui uno 0 non cambia la parità, quindi si resta fermi; un 1 la inverte, quindi si cambia stato.

Due esecuzioni passo per passo

Input	Sequenza di stati	Stato finale	Esito
0110	$q0 \to^{0} q0 \to^{1} q1 \to^{1} q0 \to^{0} q0$	q0 (accettazione)	**accettata** (due 1)
1011	$q0 \to^{1} q1 \to^{0} q1 \to^{1} q0 \to^{1} q1$	q1	**rifiutata** (tre 1)
ε (vuota)	q0 (nessuna mossa)	q0 (accettazione)	**accettata** (zero 1)

La sequenza di stati attraversati si chiama **computazione** (o *run*) dell'automa su quella stringa.

3. Definizione formale

All'esame la definizione va saputa con precisione. Un DFA è una **quintupla**

$$\mathbf{M = (Q, \Sigma, \delta, q_0, F)}$$

Componente	Che cos'è	In M_1
Q	insieme **finito** di stati	{q0, q1}
Σ	alfabeto: insieme finito di simboli in input	{0, 1}
$\delta : Q \times \Sigma \to Q$	funzione di transizione: dato lo stato attuale e il simbolo letto, dà lo stato successivo	vedi tabella sotto
$q_0 \in Q$	stato iniziale	q0
$F \subseteq Q$	insieme degli stati di accettazione (può essere vuoto)	{q0}

La funzione δ si può scrivere come **tabella di transizione**: una riga per stato, una colonna per simbolo. Per convenzione si segna con → lo stato iniziale e con * quelli di accettazione.

δ	0	1
→ * q0	q0	q1
q1	q1	q0

> **Attenzione: δ è una funzione totale.** Per *ogni* stato e *ogni* simbolo deve esistere **esattamente una** transizione. Se nel tuo disegno da uno stato manca la freccia per un simbolo, quello non è un DFA. Si risolve aggiungendo uno **stato pozzo** (o stato trappola): uno stato non di accettazione che con ogni simbolo torna su sé stesso.

Accettazione e linguaggio riconosciuto

Sia $w = w_1 w_2 \dots w_n$ una stringa su Σ. M **accetta** w se esiste una sequenza di stati $r_0, r_1, \dots, r_n$ tale che:

1. $r_0 = q_0$ (si parte dallo stato iniziale);
2. $r_i = \delta(r_{i-1}, w_i)$ per $i = 1, \dots, n$ (ogni passo segue la funzione di transizione);
3. $r_n \in F$ (si finisce in uno stato di accettazione).

Nel caso deterministico questa sequenza è unica: l'automa non ha scelte.

Il **linguaggio riconosciuto** da M è l'insieme di tutte le stringhe che accetta: **L(M) = { w ∈ Σ* | M accetta w }**. Si dice anche che M *riconosce* L(M). Nell'esempio, $L(M_1)$ = { w ∈ {0,1}* | w contiene un numero pari di 1 }.

> **Definizione.** Un linguaggio è **regolare** se esiste un DFA che lo riconosce. Tutta la prima parte del corso studia questa classe: come descriverla (DFA, NFA, espressioni regolari) e quali sono i suoi limiti (pumping lemma).

Funzione di transizione estesa

Molti testi italiani usano la funzione estesa δ*, che dice dove si arriva leggendo un'intera stringa. Si definisce per induzione sulla lunghezza:

$$\delta^*(q, \varepsilon) = q \qquad \delta^*(q, wa) = \delta(\, \delta^*(q, w), a\,)$$

Con δ* la condizione di accettazione si scrive in una riga: **w ∈ L(M) se e solo se $\delta^*(q_0, w) \in F$**. Per esempio δ*(q0, 1011) = q1 ∉ F, quindi 1011 è rifiutata.

4. Due casi limite da non confondere

Linguaggio	Cosa contiene	Come si ottiene con un DFA
∅ (linguaggio vuoto)	nessuna stringa	F = ∅, oppure nessuno stato di accettazione è raggiungibile
{ε}	solo la stringa vuota	$q_0 \in F$, e ogni simbolo porta in uno stato pozzo non di accettazione
Σ*	tutte le stringhe	tutti gli stati raggiungibili sono di accettazione

Regola pratica: **ε è accettata se e solo se lo stato iniziale è di accettazione**, perché leggendo ε l'automa non si muove.

5. Metodo per progettare un DFA

1. **Capisci cosa ricordare.** Chiediti quale informazione minima sulla parte già letta ti serve per decidere. Ogni possibile valore di quell'informazione diventa uno stato.
2. **Scegli lo stato iniziale.** Corrisponde alla situazione "non ho ancora letto niente".
3. **Disegna le transizioni.** Per ogni stato e *ogni* simbolo chiediti: "se ricordavo questo e leggo quel simbolo, cosa devo ricordare adesso?".
4. **Scegli gli stati di accettazione.** Sono quelli in cui l'informazione ricordata dice "la stringa va bene".
5. **Controlla.** Ogni stato ha una freccia per ogni simbolo? Prova almeno una stringa che deve essere accettata, una che deve essere rifiutata, e ε.

Errori frequenti all'esame

- dimenticare una transizione (δ non totale) o dimenticare lo stato pozzo;
- non controllare la stringa vuota;
- confondere "contiene la sottostringa x" con "inizia per x" o "finisce per x": sono linguaggi diversi con automi diversi;

- usare più stati del necessario va bene (l'automa resta corretto), ma uno stato che non rappresenta un'informazione chiara è spesso segno di un errore.

6. Esercizi

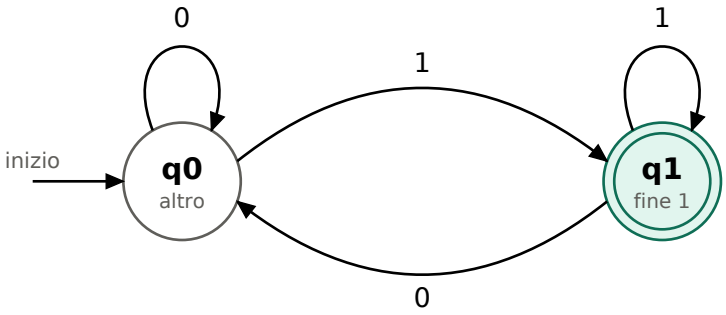
Prova a risolverli su carta prima di guardare le soluzioni nelle pagine successive. Per ogni DFA scrivi sia il disegno sia la tabella di transizione.

1. **(Facile)** Costruisci un DFA sull'alfabeto {0, 1} che accetta le stringhe che **finiscono con 1**.
2. **(Medio)** Costruisci un DFA che accetta le stringhe che **contengono la sottostringa 01**. Suggerimento: cosa deve ricordare l'automa mentre "cerca" 01?
3. **(Ragionamento)** Nel DFA M_1 della Figura 1, che linguaggio si ottiene se l'unico stato di accettazione diventa q1? E se lo sono entrambi gli stati?
4. **(Medio)** Costruisci un DFA sull'alfabeto {a, b} che accetta le stringhe la cui **lunghezza è un multiplo di 3**.
5. **(Per verificare)** Per il DFA che hai costruito nell'esercizio 2, scrivi la computazione sulle stringhe 1100 e 1001.

7. Soluzioni

Esercizio 1 — finisce con 1

Basta ricordare **l'ultimo simbolo letto**: q0 = "l'ultimo non è 1 (oppure non ho letto niente)", q1 = "l'ultimo è 1". Si accetta in q1.

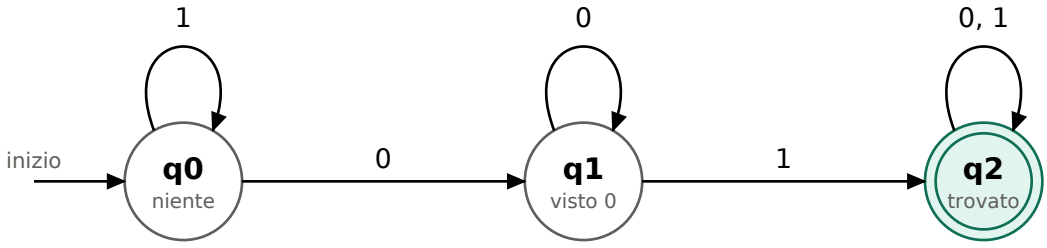


δ	0	1
→ q0	q0	q1
* q1	q0	q1

Nota: lo stato iniziale *non* è di accettazione, perché ε non finisce con 1.

Esercizio 2 — contiene 01

Servono tre informazioni: q0 = "non ho ancora iniziato a trovare 01", q1 = "ho appena letto uno 0, potrebbe essere l'inizio di 01", q2 = "ho già trovato 01". Una volta trovato 01 la stringa va bene qualunque cosa venga dopo: q2 è di accettazione e non si esce più.



δ	0	1
→ q0	q1	q0
q1	q1	q2
* q2	q2	q2

Il punto delicato è δ(q1, 0) = q1: se dopo uno 0 arriva un altro 0, l'ultimo 0 può ancora essere l'inizio di 01, quindi **non** si torna in q0.

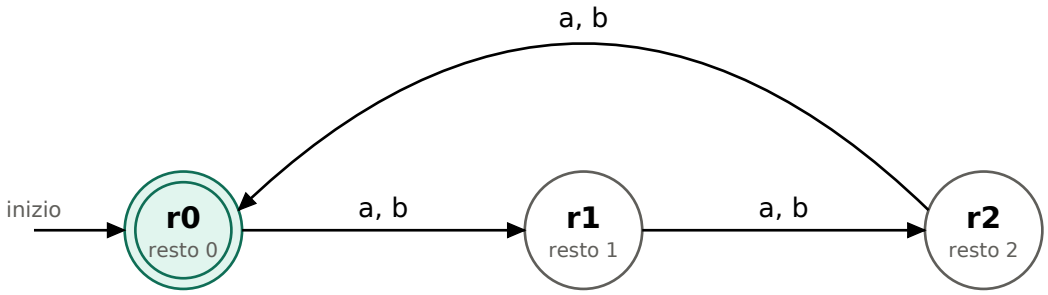
Esercizio 3 — cambiare gli stati di accettazione

- F = {q1}: il DFA accetta le stringhe con un numero **dispari** di 1. È il **complemento** del linguaggio di partenza.
- F = {q0, q1}: il DFA accetta **tutte** le stringhe, cioè Σ*.

Da ricordare: scambiare stati di accettazione e non di accettazione in un DFA dà il complemento del linguaggio. Funziona solo perché il DFA è deterministico e δ è totale: ogni stringa finisce in esattamente uno stato. È la dimostrazione che **i linguaggi regolari sono chiusi per complemento**. Con gli NFA (Lezione 2) questo trucco non funziona.

Esercizio 4 — lunghezza multipla di 3

Si ricorda il **resto della lunghezza diviso 3**: r0, r1, r2. Ogni simbolo, a o b, fa avanzare di uno il resto. Si accetta in r0, che è anche iniziale (ε ha lunghezza 0, multiplo di 3).



δ	a	b
→ * r0	r1	r1
r1	r2	r2
r2	r0	r0

Esercizio 5 — verifica sul DFA dell'esercizio 2

Input	Computazione	Esito
1100	$q0 \to^{1} q0 \to^{1} q0 \to^{0} q1 \to^{0} q1$	rifiutata (non contiene 01)
1001	$q0 \to^{1} q0 \to^{0} q1 \to^{0} q1 \to^{1} q2$	accettata (contiene 01)

Prossima lezione: operazioni regolari (unione, concatenazione, stella), chiusura dei linguaggi regolari e prime espressioni regolari.